Datacenter Town: Run, Graph and BeliefMiner Guide

12 residents, one scripted run, graphs from saved checkpoints, BeliefMiner-style scoring | Version 1, October 13, 2026 | Persona work: Shashank, Hetvi

1. What we are doing, in one page

- **Goal.** Run a town of 12 AI residents through a proposed-datacenter scenario, record what they say and remember, and measure how their causal beliefs ("datacenter → more jobs") change after events.
- **Our claim.** Known-answer testing for BeliefMiner. We write each resident's starting causal beliefs ourselves, so we have an answer key. We then check (a) whether the pipeline recovers those beliefs and (b) whether the right belief moves after the matching event.
- **Method source.** BeliefMiner (Salim, Hoque, Mueller, CSCW 2024): a fixed list of variables; people draw small cause-and-effect networks with confidence; expert networks give each link a credibility score 0–3; two measures follow: average network credibility and illusion detection. We reproduce this with AI residents in place of crowd workers. We are not plugging in the authors' software, and I have not checked whether they released code.
- **Not claimed.** 12 AI residents are not public opinion. One run is a pilot, not evidence. State this before anyone asks.
- **Order of work.** (1) Build and test the 12-resident town. (2) Run the scripted run, saving a copy of the town at fixed times. (3) Ask every resident the BeliefMiner form on each saved copy. (4) Score and draw graphs. (5) Present.

2. Status board (what is real today)

Item	Status	Evidence / note
12-resident base	Done, checked offline	All 12 load as the simulator loads them; homes, start tiles, known places, walking paths and empty memory pass.
Day-2 planning fix	In code, not run live	Ported from the Oct 6 work (plan.py). Without it each day repeats the previous plan.
Save before step 1	Done, checked offline	scratch.py no longer fails when a resident has not taken a step. All 12 save and reload.
checkpoint command, --script	Done, stub-tested, not run live	8 offline tests pass, including 'a failure drops the rest of the script'.
Scenario script (5 days)	Draft	Event wording copied from the kickoff deck, no numbers invented. Team must agree the text.
Windows fixes for the dashboard	Done	Two fixes (path splitting, process check). Uncommitted. Tell Yaksh.
Persona variable structure (~30)	Designed (section 9)	I did not find it in the repo files I changed. Confirm where it lives and in what format.
Persona validation exam	Designed (section 9)	Not built.
Topic list, answer keys, credibility table	Not started	Needs the team (section 8).
Ask-on-checkpoint script, form, scorer, graphs	Not built	Sections 7 and 8.
Any live run on the 12-resident base	Not done	No model call has been made on it yet. First live tests are in section 5.

Known: the repo's own test test_reads_run_env_line fails on Windows (it reopens a locked temp file). It is unrelated to our changes. I did not try it on a clean checkout.

3. What changed in the code (branch feature/12-residents, nothing committed)

File	Change	Why
scripts/datacenter_map.py	5 home renames added	The planner only lets a resident enter a house or room whose name holds their last name.
the_ville_datacenter map (3 CSVs)	Regenerated with the build-map command	Diff shows only the 4 new homes renamed.
scripts/build_datacenter_n12_base.py (new)	Builds the 12-resident base and its history file	Source of truth for persona text; original 8 come from the n8 script unchanged.
storage/base_the_ville_datacenter_n12 (new)	Generated base simulation	What the run forks from.
agent_history_init_datacenter_n12.csv (new)	8 original rows + 4 new rows	Each row becomes memories (thoughts) for that resident when loaded.
characters/ and profile/ (8 PNGs, new)	Sprites for the 4 new residents	The map loads a PNG named after each resident.

persona/cognitive_modules/plan.py	New day: regenerate the daily plan	Fixes the repeated-day problem.
persona/memory_structures/scratch.py	Saving with an empty clock allowed	Needed for a day-0 baseline copy.
reverie.py	'checkpoint' command; '--script file' option	Saved copies of the run; unattended scheduled events.
scripts/scenario_n12_5day.txt (new)	Draft 5-day schedule	43,200 steps, 29 checkpoints, 8 events.
translator/dashboard.py, views.py	Windows path and process-check fixes	The dashboard crashed on Windows.
README.md	Notes on the new base, command, option	

4. The 12 residents

The four new residents (one life pressure, one datacenter link, one belief)

Resident	Pressure	Link to datacenter	Seeded belief (answer-key arrow)	Event that tests it	Home / sprite
Caleb Rivera, 27, graduate, pub shifts, tutor	91 applications, 0 offers; $38k loans	Direct, may benefit	Datacenter → many jobs for locals (+)	Independent report: many temporary jobs, fewer permanent	Caleb Rivera's room, co-living space / Rajiv Patel sprite
Daniel Kim, 42, farmer	Two dry summers; ~$84k owed	Direct, may lose	Large industrial neighbor → my well drops (−)	Company revision: recycled water, clean power	Daniel Kim's apartment / Giorgio Rossi sprite
Nora Wadia, 35, librarian	Contract ends in June; ~$61k debt	Indirect, may benefit	Company tax revenue → library funding (+)	Proposal: tax incentives	Nora Wadia's house / Carmen Ortiz sprite
Sofia Alvarez, 31, grocery assistant	Hours cut to ~26/week; $6.8k card debt	Indirect, may lose	Higher power bills → fewer hours for workers (−)	Utility warning: possible rate increase	Sofia Alvarez's apartment / Tamara Taylor sprite

Profile text keeps stable facts (job, debt) in 'learned', which the code never overwrites, and puts the datacenter awareness in 'currently'. Daily-plan text is ordinary-day only and does not tell anyone to talk about the datacenter. Starting memories are three short sentences each, plus the shared proposal sentence. All dollar figures are invented scenario details.

The original eight (teammate's text, unchanged)

Maya Okonkwo (AI founder, supports), Luis Hernandez (union electrician, conditional support), Denise Brooks (shop owner, leans support), Rachel Nguyen (teacher and parent, undecided), Tom Whitaker (homeowner, leans against), Aisha Rahman (environmental advocate, conditional opposition), Marcus Williams (community organizer, opposes), Elena Chen (utility planner, evidence-dependent). Their daily-plan text still pushes datacenter talk (for example Tom: raises concerns "with anyone who will listen"). Making them richer is a separate, still-open job (section 11).

5. Before launch: the gates (do in order; every gate must pass)

Nothing below has been run live yet. The Oct 12 test (20 steps, 8 residents, gpt-4o-mini) showed: 497 model calls, $0.038, no errors; the first step took about 297 s (every resident plans the day), later steps about 0.3 s while all were asleep. No awake-time speed or cost is known.

1. **Settings.** API key in reverie/backend_server/utils.py (never share it). run_env = "staging" (cheap model gpt-4o-mini; runs are named stg-...; staging runs and their checkpoints stay out of git). Raise the spending cap on the dashboard before long tests.
2. **Rebuild and check.** From the project root:

```
python scripts/datacenter_map.py build-map
python scripts/build_datacenter_n12_base.py
```

3. **Laptop.** Plugged in; Windows sleep off when plugged in; lid close = do nothing; pause Windows updates; check free disk space. Both terminal windows must stay open. The map browser tab is not needed.
4. **Live test A (cost: cents).** Terminal 1: start the website (section 6). Terminal 2:

```
cd reverie/backend_server
python reverie.py base_the_ville_datacenter_n12 test-n12 --history
the_ville/agent_history_init_datacenter_n12.csv --run 10
```

Pass: 12 residents appear and move on /simulator_home; no errors in the terminal; cost shows on /dashboard. Then type: checkpoint (a folder stg-test-n12-ckpt-000010 appears in storage), then fin. Write down the size of that folder.

5. **Live test B (resume).** Start from the checkpoint copy and run on:

```
python reverie.py stg-test-n12-ckpt-000010 test-resume
(then at Enter option:)  run 10
```

Pass: the step counter continues from 10, not 0, and residents keep their memories.

6. **Live test C (awake hours and day change).** Run a rehearsal to about 8,700 steps (a bit past midnight of day 1 to day 2) on a test name and record: real time per game hour while awake, cost per game day, checkpoint size, and whether day-2 plans differ from day 1 (print all persona schedule). This is the number that decides how long the real run takes.
7. **Freeze.** Agree the event text. Commit the branch, record the commit id, record model and settings in the run manifest. Do not edit code during the run. Decide the run name (for example week-n12-seed1).
8. **Launch** (section 6) only when gates 1–7 pass.

6. Running the simulation

Start (two terminals)

```
Terminal 1:  cd <project>\environment\frontend_server   then   ..\..\.venv\Scripts\activate   then
python manage.py runserver
Terminal 2:  cd <project>\reverie\backend_server   then   ..\..\.venv\Scripts\activate   then
  python reverie.py base_the_ville_datacenter_n12 week-n12-seed1 --script
../../scripts/scenario_n12_5day.txt
```

- The script loads the starting memories, takes a day-0 baseline copy, then runs in segments with a copy every 6 game hours and before each event, ending with a final copy and fin.
- Watch at http://localhost:8000/dashboard (cost, step, plans). Logs: reverie/backend_server/logs/datacenter-town.log; model usage: reverie/backend_server/logs/api_usage.jsonl.
- 1 step = 10 game seconds. 360 steps = 1 hour. 8,640 = 1 day. The 5-day script is 43,200 steps; a 7-day run is 60,480.

The draft schedule (all times are game time)

Day / time	Step	Event (draft wording from the deck)	Resident it tests
Day 1	0–8,640	No events. Baseline day. Copy at step 0 and every 2,160 steps.	All: day-0 baseline
Day 2, 09:00	11,880	Proposal filed: tax incentives and projected construction jobs	Nora
Day 2, 15:00	14,040	Independent report: many temporary jobs, fewer permanent roles	Caleb
Day 3, 09:00	20,520	Utility warning: possible household rate increase	Sofia
Day 3, 15:00	22,680	Company revision: recycled water and dedicated clean power	Daniel
Day 4, 09:00	29,160	External shock: heat wave, then a grid emergency	All
Day 4, 15:00	31,320	Trust shock: a leaked memo conflicts with an earlier company claim	All
Day 5, 09:00	37,800	Social signal: union endorsement and organized opposition	All
Day 5, 15:00	39,960	Negotiation: enforceable community benefits agreement	All

Every event is delivered to all residents at once (call -- whisper all). That is exposure by assignment, which is fine for a controlled design but not how news spreads. A copy is taken just before each event, so the clean pre-event state is always available. Longer runs: add days and events to the script; the generator logic is simple (stop points = every 2,160 steps plus event steps).

If something goes wrong

- **Run stops (spending cap, stop button, a failing command):** the rest of the script is skipped on purpose so no event fires at the wrong step. Raise the cap, then resume (below).
- **Laptop slept or crashed:** autosave is every 360 steps. Resume from the newest checkpoint copy (loses at most 6 game hours).
- **Resume:** start reverie.py with the checkpoint folder name as the fork and a new run name, and a copy of the script with the already-finished lines deleted. The new run inherits the earlier logs.

7. What the run produces

Output	Where	What it holds / limit
Run folder	environment/frontend_server/storage/stg-<name>/	Latest saved state. Each save overwrites the previous one, so it is NOT a history.
Checkpoint copies	storage/stg-<name>-ckpt-<step>/	Full memory and state at that step (no movement files). These are the history. About 29 for the 5-day script.
conversations.jsonl	<run>/experiment/	Every resident-to-resident conversation (use for the talk network and for dialogue evidence).
events.jsonl	<run>/experiment/	History loading, each event whisper, each checkpoint marker. This is the exposure record.
agent_snapshots.jsonl	<run>/experiment/	Every 50 steps: status text, current action, place, chat partner. Thin: no memories, no beliefs.
run_manifest.json, outcomes.jsonl	<run>/experiment/	Settings and end-of-save status text.

Per-step movement	<run>/movement/	Positions per step (only in the live run folder, not in checkpoint copies).
Cost and usage	reverie/backend_server/logs/api_usage.jsonl	Calls, tokens and cost per run.

Key point: beliefs are not logged during the run. They exist only inside each resident's memory, so the checkpoint copies are the only way to ask 'what did they believe on day 3?' later.

8. BeliefMiner integration, step by step

1. **Topic list (shared nodes).** 12–15 topics, one name and one definition each (for example: datacenter built, local jobs, electricity bills, water use, tax revenue, property value, noise, trust in company). Every resident uses the same list. Deliverable: a small JSON file. Owner: to decide.
2. **Seeded-belief key (what we gave each resident).** One short list of arrows per resident: cause, effect, direction (+/−), strength, starting confidence, and a flag if it is a possible illusion. The four new residents are already designed (section 4). The original eight need arrows drafted from their profile text and confirmed by whoever wrote them.
3. **Expert credibility table (BeliefMiner ground truth).** The paper (section 4.1.2) uses at least three independent experts who each draw the network with references, then meet and score each link 0–3 (3 = all included it, 0 = none). We have no domain experts; options: three teammates build it independently from cited sources; use the agreed event fact sheet; or ask the professor's lab. Score only the links that matter (about 30–50), others default to 0.
4. **The form.** For each resident, ask them to pick links from the topic list (cause, effect, more/less, strength, how sure) and give one sentence of reasons. A structured form, not free chat, is the safer start; reading arrows out of free chat is an optional extra to compare.
5. **Ask on the saved copies, quietly (new script, not built).** A script loads each resident from a checkpoint folder, retrieves memories without updating their 'last accessed' time (the main branch's retrieve function always updates it; Mihir's branch has a touch=False option), asks the form once, and saves the answer as JSON. It never writes into the live run. Use a different model for scoring than for the residents.
6. **Test on made-up answers first.** Write answers with known arrows, including negation, 'if/then' conditions, reported speech, and statements with no causal claim. Decide the pass bar before looking at results.
7. **Day-0 fidelity check.** Ask the form on the day-0 copy. Compare with the seeded key. If a resident does not hold the arrow we gave, fix the persona or the form before reading any later graph.
8. **Run on every checkpoint.** 12 residents × about 29 copies = about 350 asks.
9. **Score (per resident, per copy).** Average network credibility = mean credibility of the arrows they drew. Illusion flags per arrow: believed more strongly than its credibility supports ('potentially misinformed'), or less strongly than it supports. Change = difference from the day-0 key and from the previous copy.
10. **Human audit.** Two teammates review a random sample of drawn arrows against the quoted sentence.

Limits to state: AI residents are an artificial 'crowd' of 12, so report per resident and per link, not crowd statistics; answers depend on the model; one run is a pilot.

9. Persona variables and the persona check (planned structure)

Today each resident has 47 variables in scratch.json, but only 7 describe the person (age, innate, learned, currently, lifestyle, living_area, daily_plan_req) and they are free text. The other 40 are memory settings (17) and runtime state (20), plus name fields. The professor asked for a better structure with more useful variables. Proposed: about 30 more.

Group	Variables
Situation	job, employment status, income level, debt level, own or rent, household, years in town
Exposure	distance from site, ways it could help, ways it could hurt, direct or indirect stake
Information and trust	prior knowledge, main sources, trust in company, council, utility, experts (1–5)
Reasoning style	reasoning depth (1–5), evidence needed to change their mind, openness, certainty of starting view
Values	ranked priorities (jobs, living costs, environment, fair process, growth)
Social	talkative or reserved, peer influence, who they are close to
Personality and stance	Big Five traits; starting position on a 5-level scale (the professor's messaging paper uses both; check its exact scale)
Seeded beliefs	the answer-key arrows from section 8, step 2

- Mechanism: the simulator only reads profile text. A variable changes behavior only if a build step writes it into that text; the rest are for analysis and tests. Mark each variable 'in prompt' or 'measure only'.

- Persona check (add to the test): ask each resident to report each variable (for example 'how much do you trust the utility, 1 to 5?') and compare with the assigned value. Add one or two behavior probes with an expected pattern per variable (heavy debt → mentions money pressure; low trust in the utility → does not accept a utility warning at face value). Include trick questions about facts they were never told (for example datacenter water use): the right answer is 'I don't know'. Score the match rate; expect some drift after day 1 because the code rewrites 'currently' and the daily plan each new day.

10. Graphs and expected outcomes

Graph	Built from	What it shows	Reads as
G1 Belief maps	Form answers per copy	Per resident: topics as dots, arrows with direction, strength and confidence; arrow colour = credibility 0–3. Small multiples for the four test residents across days.	Which beliefs a resident holds, and which are weakly supported.
G2 Score timeline	Scores per copy	Line per resident of average network credibility over time, with a vertical line at each event; count of illusion flags.	Do beliefs get better or worse supported, and when.
G3 Day-0 recovery	Day-0 form vs seeded key	Per resident: seeded arrows found, missed, extra.	Whether the pipeline can be trusted at all.
G4 Event response	Copies before and after each event	Table or heat map: rows residents, columns events, cell = change in the tested arrow's strength.	Did the matching event move the matching belief.
G5 Talk network (optional)	conversations.jsonl	Who talked to whom per day.	Whether beliefs could spread by conversation.

Hypotheses (to test, not results)

- H1 Caleb: after the independent report (mostly temporary jobs), his 'datacenter → jobs for locals' arrow weakens.
- H2 Daniel: after the recycled-water revision, his 'datacenter → my well drops' arrow weakens or becomes conditional, unless his distrust holds it in place.
- H3 Nora: after the tax-incentive proposal, her 'tax revenue → library funding' arrow weakens.
- H4 Sofia: after the utility warning, her 'power bills → fewer hours' arrow strengthens and she starts linking the datacenter to bills.
- Also report cases where the stance (or what they say) changes but the arrows do not, and the reverse. A null or messy result is reportable; say so honestly.

What decides success: G3 must be acceptable first. If day-0 recovery is poor, fix the persona or the form before interpreting G2 and G4.

11. Plan to Tuesday, October 20 (targets, not promises)

When	Work
Tue Oct 13	Agree variable structure and event wording. Draft arrows for the original 8. Start topic list. Write the ask-on-checkpoint script.
Wed Oct 14	Gates 1–6 (live tests A–C). Credibility table built independently by three people. Test the form on made-up answers.
Thu Oct 15	Freeze and launch the scripted run if all gates pass. If a gate fails, fix it and launch next day: a late start costs little, a broken run costs the week.
Fri–Sat	Monitor. Build the scorer and graph code on the copies that already exist.
Sun Oct 18	Ask the form on all copies. Human audit sample.
Mon Oct 19	Graphs G1–G4, one-page summary, slides.
Tue Oct 20	Present: what we ran, three figures, day-0 check, limits, next step.

Real run time is unknown until gate 6. If awake steps are slow, shorten the script (fewer days) and keep the checkpoint and event timing.

12. Risks and open decisions

- **Run naming and git:** prod- runs and their checkpoint folders are tracked in git (about 29 big copies). Staging (stg-) keeps them local and uses the cheap model the professor asked for. Decide before launch.
- **Disk:** checkpoint size is unknown until gate 4. Check free space.
- **Event delivery:** every event reaches everyone instantly. Say so in the write-up, or later add a slower channel.
- **Original eight:** their profiles still push datacenter talk and are shorter than the new four. Upgrade them, or note the difference.

- **Questionnaire owner:** agree the first question version and output format now; label later changes v2 so the baseline stays comparable.
- **Model variation:** the same checkpoint can give different answers on a second ask. Ask twice on a few copies to measure this.
- **Team overlap:** Mihir's branch (stance line graph, memory-safe probe) is separate from ours; agree who merges what so the retrieve function is not changed twice.
- **Unverified items:** every live behavior in this guide (new base, plan fix, scripted run, resume) is untested until gates 4–6 pass.